

Important Thread Methods in Java

Java provides several methods for controlling, coordinating, inspecting, and configuring threads.

The most important methods covered here are:

```
sleep()  
join()  
yield()  
interrupt()  
isInterrupted()  
interrupted()  
isAlive()  
currentThread()  
setName()  
getName()  
setPriority()  
setDaemon()
```

Each method affects a thread differently, so it is important to understand:

- Which thread is affected
- Whether the thread blocks
- Whether a lock is released
- Which thread state is entered
- Whether the behaviour is guaranteed or only suggested

Thread.sleep()

1. What Does `sleep()` Do?

`Thread.sleep()` pauses the currently executing thread for a specified duration.

```
Thread.sleep(milliseconds);
```

Example:

```
public class Main {

    public static void main(String[] args) {
        System.out.println("Start");

        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        System.out.println("End");
    }
}
```

The main thread pauses for approximately two seconds before printing `End`.

2. Which Thread Sleeps?

`sleep()` is a static method of the `Thread` class.

```
Thread.sleep(2000);
```

The thread that executes this statement is the thread that sleeps.

For example:

```
Thread t1 = new Thread(() -> {
    try {
        Thread.sleep(2000);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});
```

Here, `t1` sleeps because `t1` executes the method call.

Calling `sleep()` through a thread reference does not make that referenced thread sleep:

```
t1.sleep(2000);
```

Although this compiles, it still pauses the current thread, not necessarily `t1`.

Therefore, always write:

```
Thread.sleep(2000);
```

3. Thread State During `sleep()`

While sleeping, the thread enters:

```
TIMED_WAITING
```

Conceptually:

```
Executing  
→ TIMED_WAITING  
→ eligible to run again
```

After the sleep duration ends, the thread becomes runnable again. It does not necessarily resume immediately because the scheduler must select it.

4. Does `sleep()` Release a Lock?

No.

If a thread calls `sleep()` while holding a monitor, it continues holding that monitor.

```
synchronized (lock) {  
    Thread.sleep(2000);  
}
```

During those two seconds:

```
Current thread is sleeping  
+
```

current thread still owns lock

Other threads that require the same monitor remain blocked.

This is an important difference between:

```
Thread.sleep()
```

and:

```
wait()
```

`wait()` releases the monitor on which it is called, while `sleep()` does not.

5. `InterruptedException` with `sleep()`

`sleep()` throws `InterruptedException`.

```
try {
    Thread.sleep(2000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

If another thread interrupts a sleeping thread:

```
thread.interrupt();
```

the sleeping thread wakes early and `sleep()` throws `InterruptedException`.

When the exception is thrown, the interrupted status is cleared. If the method cannot fully handle the interruption, it commonly restores the status:

```
Thread.currentThread().interrupt();
```

6. Important Points About `sleep()`

1. It pauses the currently executing thread.

2. It normally places the thread in `TIMED_WAITING`.
3. It does not release monitors held by the thread.
4. It does not guarantee an exact wake-up time.
5. It may return later than requested because of scheduling.
6. It throws `InterruptedException` if the sleeping thread is interrupted.

Thread.join()

7. What Does `join()` Do?

`join()` makes the current thread wait for another thread to finish.

```
targetThread.join();
```

The calling thread waits. The target thread continues running.

Example:

```
public class Main {

    public static void main(String[] args)
        throws InterruptedException {

        Thread t1 = new Thread(() -> {
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }

            System.out.println("t1 finished");
        });

        t1.start();
    }
}
```

```
t1.join();

System.out.println("Main finished");
}
}
```

Output:

```
t1 finished
Main finished
```

The main thread waits at:

```
t1.join();
```

until `t1` terminates.

8. Which Thread Waits During `join()` ?

Consider:

```
t1.join();
```

This means:

```
Current thread waits
until
t1 terminates
```

If the main thread executes that statement:

```
Main thread waits
t1 keeps running
```

A common misunderstanding is that `join()` pauses the thread on which it is called. It does not.

It pauses the thread that calls `join()`.

9. Thread State During `join()`

For an untimed join:

```
t1.join();
```

the calling thread generally enters:

WAITING

Conceptually:

```
Main thread waits  
→ t1 finishes  
→ main thread becomes runnable again
```

The target thread eventually enters:

TERMINATED

10. Timed `join()`

Java provides timed versions:

```
t1.join(2000);
```

This means:

| Wait until `t1` finishes, but for no longer than approximately two seconds.

The calling thread enters:

TIMED_WAITING

It may continue because:

- The target thread finished
- The timeout expired

- The calling thread was interrupted

Timed `join()` does not terminate the target thread when the timeout expires.

11. Controlling Execution Order with `join()`

`join()` can enforce partial ordering.

```
t1.start();
t1.join();

t2.start();
```

This guarantees:

```
t1 completes
before
t2 is started
```

However, excessive use of `join()` can remove concurrency by forcing threads to run sequentially.

12. Important Points About `join()`

1. The calling thread waits.
2. The target thread continues running.
3. Untimed `join()` normally places the caller in `WAITING`.
4. Timed `join()` normally places the caller in `TIMED_WAITING`.
5. `join()` throws `InterruptedException`.
6. A timed join does not stop the target thread.
7. `join()` creates a happens-before relationship: actions in the completed thread become visible to the thread that successfully returns from `join()`.

`Thread.yield()`

13. What Does `yield()` Do?

`Thread.yield()` is a hint to the scheduler that the current thread is willing to give other runnable threads an opportunity to execute.

```
Thread.yield();
```

Example:

```
public class Main {

    public static void main(String[] args) {

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                System.out.println("T1: " + i);
                Thread.yield();
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                System.out.println("T2: " + i);
            }
        });

        t1.start();
        t2.start();

    }
}
```

The exact output order is unpredictable.

14. `yield()` Is Only a Hint

The scheduler may:

- Run another thread
- Continue running the same thread

- Ignore the hint completely

Therefore:

```
yield() does not guarantee a context switch
```

It should never be used to enforce program correctness or thread ordering.

15. Thread State During `yield()`

`yield()` does not place the thread in:

```
WAITING
```

or:

```
TIMED_WAITING
```

The thread remains runnable and eligible for execution.

Conceptually:

```
Running thread  
→ gives scheduler another scheduling opportunity  
→ may continue or may be replaced
```

16. When Is `yield()` Used?

`yield()` is rarely needed in production application code.

It may be useful for:

- Learning thread scheduling
- Experiments
- Testing unusual scheduling behaviour
- Very specialized low-level code

It should not be used as a replacement for:

- Synchronization
- Locks
- `join()`
- `wait()` and `notifyAll()`
- Executors
- Coordination utilities

Thread.interrupt()

17. What Does `interrupt()` Do?

`interrupt()` requests that a thread stop or change what it is currently doing.

```
thread.interrupt();
```

It does not forcibly kill the target thread.

Instead, interruption is a cooperative cancellation mechanism.

```
One thread requests interruption
→ target thread detects the request
→ target thread decides how to respond
```

18. Interrupting a Running Thread

Consider:

```
Thread t = new Thread(() -> {
    while (true) {
        System.out.println("Running...");
    }
});

t.start();
```

```
t.interrupt();
```

The thread may continue running forever because its code never checks the interrupted status.

Calling:

```
t.interrupt();
```

does not automatically terminate the loop.

19. Interrupted Status

Each thread has an interrupted status.

Conceptually:

```
Not interrupted → false  
Interrupted      → true
```

When another thread calls:

```
t.interrupt();
```

and `t` is actively running, its interrupted status is normally set to `true`.

The target thread must inspect that status.

20. Cooperative Cancellation

A common pattern is:

```
Thread worker = new Thread(() -> {  
    while (!Thread.currentThread().isInterrupted()) {  
        System.out.println("Working...");  
    }  
})
```

```
        System.out.println("Stopped gracefully");
    });
```

Another thread can request cancellation:

```
worker.interrupt();
```

The worker notices the interrupt request and exits its loop cleanly.

21. Interrupting `sleep()`, `wait()`, or `join()`

If a thread is blocked in an interruptible method such as:

```
Thread.sleep()
Object.wait()
Thread.join()
```

and another thread interrupts it:

```
thread.interrupt();
```

the blocking method throws:

```
InterruptedException
```

Example:

```
Thread worker = new Thread(() -> {
    try {
        Thread.sleep(10_000);
    } catch (InterruptedException e) {
        System.out.println("Sleep interrupted");
        Thread.currentThread().interrupt();
    }
});
```

The thread does not need to wait for the full sleep duration.

22. Interrupt Status After `InterruptedException`

When `sleep()`, `wait()`, or `join()` throws `InterruptedException`, the interrupted status is cleared.

Therefore, this code:

```
catch (InterruptedException e) {
    System.out.println(
        Thread.currentThread().isInterrupted()
    );
}
```

normally prints:

```
false
```

If higher-level code must know that interruption occurred, restore the status:

```
catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

23. Correct Ways to Handle `InterruptedException`

Propagate the Exception

```
void performTask() throws InterruptedException {
    Thread.sleep(1000);
}
```

Restore the Interrupt Status

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

```
    return;  
}
```

Perform Cleanup and Stop

```
try {  
    processTask();  
} catch (InterruptedException e) {  
    cleanup();  
    Thread.currentThread().interrupt();  
}
```

Silently ignoring interruption is usually a mistake:

```
catch (InterruptedException e) {  
}
```

24. `isInterrupted()` vs `interrupted()`

These methods both inspect interrupted status, but they behave differently.

`isInterrupted()`

```
thread.isInterrupted();
```

It:

- Checks the interrupted status of a particular thread
- Returns `true` or `false`
- Does not clear the status

Example:

```
boolean interrupted = worker.isInterrupted();
```

`Thread.interrupted()`

```
Thread.interrupted();
```

It:

- Checks the interrupted status of the current thread
- Is a static method
- Clears the current thread's interrupted status after reading it

Example:

```
boolean first = Thread.interrupted();  
boolean second = Thread.interrupted();
```

If the status was initially `true`:

```
first → true  
second → false
```

The first call cleared it.

25. Comparison

Method	Checks Which Thread?	Clears Status?
<code>thread.isInterrupted()</code>	The referenced thread	No
<code>Thread.interrupted()</code>	The current thread	Yes

26. Why Java Uses Cooperative Cancellation

Forcibly stopping a thread can leave shared state inconsistent.

The thread may be terminated while:

- Holding a lock
- Updating a collection
- Transferring money
- Writing a file

- Modifying several related fields

Cooperative interruption allows the thread to:

- Finish or abandon work safely
- Release resources
- Restore invariants
- Exit at a controlled point

Deprecated methods such as `Thread.stop()` should not be used.

27. Common Uses of Interruption

Interruption is used for:

- Cancelling long-running tasks
- Stopping worker threads
- Shutting down thread pools
- Ending blocking operations
- Responding to application shutdown
- Cancelling futures and executor tasks

`isAlive()`

28. What Does `isAlive()` Do?

`isAlive()` checks whether a thread has been started and has not yet terminated.

```
thread.isAlive();
```

Example:

```
public class Main {

    public static void main(String[] args)
        throws InterruptedException {
```

```
Thread worker = new Thread(() -> {
    try {
        Thread.sleep(2000);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});

System.out.println(worker.isAlive());

worker.start();

System.out.println(worker.isAlive());

worker.join();

System.out.println(worker.isAlive());
}
```

Output:

```
false
true
false
```

29. Meaning of the Result

Before `start()`:

```
isAlive() → false
```

After `start()` and before termination:

```
isAlive() → true
```

After the thread terminates:

```
isAlive() → false
```

`isAlive()` does not tell whether the thread is currently using the CPU. A thread may be sleeping, waiting, blocked, or runnable and still be alive.

Thread.currentThread()

30. What Does `currentThread()` Do?

`Thread.currentThread()` returns a reference to the thread currently executing the code.

```
Thread current = Thread.currentThread();
```

Example:

```
public class Main {  
  
    public static void main(String[] args) {  
        Thread current = Thread.currentThread();  
  
        System.out.println(current.getName());  
    }  
}
```

The output is usually:

```
main
```

31. Why `currentThread()` Is Useful

Inside a task, the code may not have direct access to the original thread variable.

```
Runnable task = () -> {  
    Thread current = Thread.currentThread();
```

```
System.out.println(current.getName());  
};
```

Common uses include:

- Logging
- Debugging
- Reading the current thread name
- Checking interrupted status
- Restoring interrupted status
- Inspecting thread priority
- Identifying which worker executed a task

Thread Naming

32. Setting a Thread Name

A thread name can be assigned using `setName()`:

```
Thread worker = new Thread(() -> {  
    System.out.println(  
        Thread.currentThread().getName()  
    );  
});  
  
worker.setName("Worker-1");  
worker.start();
```

A cleaner approach is to pass the name to the constructor:

```
Thread worker = new Thread("Worker-1" -> {  
    System.out.println(  
        Thread.currentThread().getName()  
    );  
});
```

```
} , "Worker-1");  
  
worker.start();
```

33. Reading a Thread Name

Use:

```
thread.getName();
```

or inside the running task:

```
Thread.currentThread().getName();
```

34. Why Thread Names Matter

Meaningful thread names improve:

- Log readability
- Debugging
- Stack-trace analysis
- Thread-dump analysis
- Production monitoring
- Identification of worker responsibilities

Compare:

```
Thread-7 failed
```

with:

```
payment-worker-3 failed
```

The second message provides much more context.

Thread Priority

35. What Is Thread Priority?

Thread priority is a scheduling hint that indicates the relative importance of a thread.

Java defines priorities from 1 to 10.

```
Thread.MIN_PRIORITY    // 1
Thread.NORM_PRIORITY   // 5
Thread.MAX_PRIORITY    // 10
```

The default priority is normally:

```
Thread.NORM_PRIORITY
```

A new thread generally inherits the priority of the thread that creates it.

36. Setting Priority

```
Thread worker = new Thread(() -> {
    System.out.println("Working");
});

worker.setPriority(Thread.MAX_PRIORITY);
worker.start();
```

Example:

```
Thread lowPriority = new Thread(() -> {
    System.out.println("Low priority");
});

lowPriority.setPriority(Thread.MIN_PRIORITY);

Thread highPriority = new Thread(() -> {
    System.out.println("High priority");
});
```

```
highPriority.setPriority(Thread.MAX_PRIORITY);

lowPriority.start();
highPriority.start();
```

The high-priority thread is not guaranteed to execute first.

37. Priority Does Not Guarantee Execution Order

The operating system and JVM ultimately influence scheduling.

Different platforms may:

- Respect priorities differently
- Map several Java priorities to the same native priority
- Partially ignore priority differences
- Produce different execution orders

Therefore:

```
Higher priority ≠ guaranteed first execution
Higher priority ≠ guaranteed more CPU time
```

Never use thread priority to guarantee program correctness.

38. When Should Priority Be Used?

Thread priority is rarely needed in normal application code.

Possible specialized uses include:

- Low-level performance tuning
- Background maintenance work
- Platform-specific systems
- Carefully measured real-time-like workloads

In most applications, use:

- Executors

- Separate thread pools
- Queue policies
- Task prioritization
- Proper concurrency utilities

These mechanisms provide more predictable control than raw thread priorities.

Daemon Threads

39. What Is a Daemon Thread?

A daemon thread is a background-support thread that does not keep the JVM alive by itself.

```
thread.setDaemon(true);
```

The JVM may terminate when no live non-daemon threads remain.

40. User Threads and Daemon Threads

User Thread

A normal thread is a non-daemon thread.

```
JVM remains alive while a user thread is running
```

Examples:

- Main application logic
 - Business operations
 - Request processing
 - Important file or database work
-

Daemon Thread

A daemon thread performs background-support work.

JVM does not remain alive solely for daemon threads

Examples may include:

- Runtime maintenance threads
- Monitoring helpers
- Cache cleanup
- Background housekeeping

The exact thread type used by a library or JVM implementation may vary.

41. Daemon Thread Example

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Thread daemon = new Thread(() -> {  
            while (true) {  
                System.out.println("Daemon running...");  
            }  
        });  
  
        daemon.setDaemon(true);  
        daemon.start();  
  
        System.out.println("Main finished");  
    }  
}
```

Once the main thread finishes, the JVM may exit because only the daemon thread remains.

The daemon thread is not allowed to keep the process alive.

42. Important Daemon Rule

A thread must be marked as daemon before it is started.

Correct:

```
Thread worker = new Thread(task);  
  
worker.setDaemon(true);  
worker.start();
```

Incorrect:

```
worker.start();  
worker.setDaemon(true);
```

Calling `setDaemon()` after the thread has started throws:

```
IllegalThreadStateException
```

43. Daemon Threads Can Stop Abruptly

When the JVM exits because only daemon threads remain, daemon threads may be terminated without completing their work.

They are not guaranteed to execute:

- Remaining statements
- Cleanup code
- `finally` blocks
- File flushes
- Database commits
- Network operations

Therefore, daemon threads should not perform work that must complete reliably.

Do not rely on a daemon thread for:

- Saving critical data
- Completing financial transactions

- Writing essential files
- Sending guaranteed messages
- Releasing important external resources

44. JVM Exit Rule

The practical rule is:

The JVM continues running while at least one live non-daemon thread exists.

When the last non-daemon thread finishes:

```
Only daemon threads remain
→ JVM may terminate
```

The main thread itself is normally a non-daemon thread.

Comparison of Important Methods

Method	Affects Which Thread?	Typical State	Releases Monitor?	Main Purpose
<code>Thread.sleep()</code>	Current thread	<code>TIMED_WAITING</code>	No	Pause for a duration
<code>thread.join()</code>	Calling thread	<code>WAITING</code>	Not a general lock-release mechanism	Wait for another thread to finish
<code>thread.join(timeout)</code>	Calling thread	<code>TIMED_WAITING</code>	Not a general lock-release mechanism	Wait with a timeout
<code>Thread.yield()</code>	Current thread	Remains runnable	No	Scheduler hint
<code>thread.interrupt()</code>	Target thread	Depends on target state	No direct guarantee	Request cancellation
<code>thread.isAlive()</code>	Inspects target thread	No state change	No	Check whether thread is alive

Method	Affects Which Thread?	Typical State	Releases Monitor?	Main Purpose
<code>Thread.currentThread()</code>	Returns current thread	No state change	No	Access executing thread

sleep() vs join() vs yield()

Feature	<code>sleep()</code>	<code>join()</code>	<code>yield()</code>
Main purpose	Pause for time	Wait for target thread	Suggest rescheduling
Thread affected	Current thread	Calling thread	Current thread
Guaranteed waiting	Yes, unless interrupted	Yes, until completion/timeout/interruption	No
State	<code>TIMED_WAITING</code>	<code>WAITING</code> or <code>TIMED_WAITING</code>	Runnable
Releases held monitors	No	No general release of monitors held by caller	No
Throws <code>InterruptedException</code>	Yes	Yes	No

Important Rules

1. `Thread.sleep()` always pauses the currently executing thread.
2. `sleep()` does not release a monitor held by the sleeping thread.
3. `join()` pauses the calling thread, not the target thread.
4. `yield()` is only a scheduling hint and may be ignored.
5. `interrupt()` does not forcibly terminate a thread.
6. Interruption requires cooperation from the target task.
7. Interrupting `sleep()`, `wait()`, or `join()` causes `InterruptedException`.
8. `InterruptedException` clears the interrupted status.
9. Restore the status when interruption cannot be fully handled.
10. `isInterrupted()` does not clear the status.

11. `Thread.interrupted()` checks and clears the current thread's status.
 12. `isAlive()` is true after start and before termination.
 13. Thread names are important for logs and debugging.
 14. Thread priority does not guarantee execution order.
 15. Set daemon status before calling `start()`.
 16. Daemon threads should not be used for work that must finish reliably.
 17. The JVM can exit when only daemon threads remain.
-

Final Summary

```
sleep()
→ pause the current thread for a duration

join()
→ make the current thread wait for another thread

yield()
→ suggest that the scheduler consider another runnable thread

interrupt()
→ request cooperative cancellation

isInterrupted()
→ inspect status without clearing it

Thread.interrupted()
→ inspect and clear the current thread's status

isAlive()
→ check whether a thread has started and not terminated

currentThread()
→ obtain the currently executing thread

setName()
→ assign a meaningful thread name

setPriority()
→ provide a non-guaranteed scheduling hint

setDaemon()
→ mark a thread as background support work
```

These methods provide low-level thread control. In larger applications, thread creation and lifecycle management are usually handled through higher-level APIs such as executors, futures, concurrent collections, and structured task coordination.

Coder Army